

FalconVQA

Fast Augmented Language-based CONversational
Video Question Answering

A Semantic Retrieval Layer for Surveillance Footage
Software Design Document

Avinash Anish

Vaivaswat Verma

August 6, 2026

Contents

1	Introduction and Problem Understanding	2
1.1	System Identification	2
1.2	Problem Statement	2
1.3	Project Objectives	3
1.4	Document Organisation	4
2	Solution Overview	4
2.1	Partitioning the Recording	4
2.2	Analysing Each Chunk	5
2.3	Indexing and Aggregation	5
2.4	Search and Question Answering	5
3	Requirements Analysis	6
3.1	Functional Requirements	6
3.2	Non-Functional Requirements	6
4	Creative Enhancements	8
5	Detailed Design of the Processing Pipeline	10
5.1	Stage 1: Chunking	10
5.2	Stage 2: Analysis	11
5.3	Stage 3: Indexing and Aggregation	14
5.4	Stage 4: Retrieval and Question Answering	16
6	Architecture and Technology Stack	18
6.1	Design Methodology	18
6.2	Architectural Overview	18
6.3	Module Inventory	19
6.4	Ingestion Lifecycle	19
6.5	Persistence	19
6.6	Technology Stack	20
6.7	System Requirements	21
6.8	Known Limitations	21
6.9	Scope for Future Enhancements	22

7	Database Design	23
7.1	Storage Strategy	23
7.2	Application Database	23
7.3	Object Storage	25
7.4	Analysis Record Store	25
7.5	Vector Store	26
7.6	Cross-Store Identity	27
8	User Interface Design	28
9	Acceptance Test Cases and Test Plan	30
9.1	Acceptance Criteria	30
9.2	Case Format and Severity	32
9.3	Test Cases: Accounts and Tenancy	32
9.4	Test Cases: Ingestion and Processing	32
9.5	Test Cases: Search and Filtering	34
9.6	Test Cases: Question Answering and Conversation	34
9.7	Test Cases: Review, Playback and Inspection	35
9.8	Test Cases: Programmatic Access	37
9.9	Non-Functional Verification	37
9.10	Test Environment	37
9.11	Test Data	38
9.12	Entry, Exit and Suspension	39
9.13	Test Automation	39
9.14	Defect Recording and Reporting	40
10	Execution Plan	41
10.1	Schedule	41
10.2	Distribution of Work	41

1 Introduction and Problem Understanding

1.1 System Identification

This document specifies the design of **FalconVQA**, short for Fast Augmented Language-based CONversational Video Question Answering. It is a retrieval and question answering system for recorded video, developed with surveillance footage as its primary application domain. The name records the four properties the system is designed around: retrieval that is **fast** enough to be interactive, an index **augmented** beyond the raw signal with structured descriptions produced by vision and audio models, a **language-based** interface in which both the index and the query are natural language, and a **conversational** access path in which an operator or an autonomous agent asks questions of a recording and receives cited answers rather than a list of files.

1.2 Problem Statement

A natural language front end that extracts the relevant footage from a given recorded video by description alone.

Background

Surveillance footage is generally recorded onto network video recorders and other digital storage devices. Finding a particular piece of footage is carried out by entering the date and time of the incident and reviewing what comes back, which is slow. The recording exists as a continuous timeline indexed by nothing but a clock, so the date and time are the only handles the operator has.

That procedure works only when the time of the incident is already known. In practice it usually is not. What is available instead is a description of the event, obtained from a witness or from a report: a man in a yellow shirt left a bag near the checkout. No stage of the recording chain produces any account of what the footage contains, so a description of this kind cannot be used as a query at all.

Challenge

To develop a simple natural language front end in which the operator types a description of the event into a prompt box, and the matching footage is extracted and played back.

Two things stand between the description and the playback. The first is that the footage carries no description to match against, so one has to be derived. The second is that general-purpose video understanding tools do not derive it, because they are built for edited media and rely on three properties surveillance footage does not have:

- **Shot boundaries.** A fixed camera produces none.
- **Speech.** A substantial proportion of surveillance footage is silent.
- **Visually salient events.** Events of interest are typically brief and visually unremarkable.

Applied to such footage, these tools either return the entire recording as a single segment or subdivide it into a large number of near-identical segments. Neither result is searchable.

The problem this project addresses is therefore twofold: derive a descriptive representation of footage that has little intrinsic structure, and make the footage retrievable through that representation. The operator states what is being looked for in plain language and receives the corresponding moment, timestamped and ready to play.

1.3 Project Objectives

The system is required to deliver the following:

- O-1. A processing pipeline that accepts a recording and renders it searchable by description.
- O-2. Semantic search returning ranked moments with timecodes, and question answering that returns a synthesised answer with references to the moments it was derived from.
- O-3. Video-level output that can be read in place of watching the footage: a summary, chapters, notable events, the people and objects that appear, and any anomalies.
- O-4. A multi-tenant web application in which uploading, searching, questioning and playback are performed in a single workspace, with each operator's recordings isolated from every other operator's.
- O-5. A documented and self-describing HTTP API, so that the same capabilities can be driven by an LLM agent rather than only by a human operator.

1.4 Document Organisation

Section 2 presents the solution and the processing pipeline in outline. Section 3 states the functional and non-functional requirements. Section 4 records the design enhancements that distinguish the system from a conventional caption-and-embed pipeline. Section 5 is the detailed design: each of the four pipeline stages, the models employed, the parameters governing them and the outputs produced. Section 6 specifies the architecture and technology stack, and Section 7 the database design across all four stores. Section 8 describes the user interface. Section 9 states the acceptance criteria and the test cases that establish them, followed by the plan under which those cases are run: the environment, the test data, the entry and exit criteria, and what is automated. Section 10 sets out the execution schedule and the distribution of work.

2 Solution Overview

FalconVQA processes a recording in five stages: the video is partitioned into chunks, each chunk is analysed, the resulting records are indexed, video-level aggregates are derived from those records, and the indexed material is then queried by search or by question. Most stages are parameterised rather than fixed, because surveillance footage varies too widely for a single configuration to be appropriate in all cases. This section states what each stage does; Section 5 specifies the models, parameters and outputs of each in full.

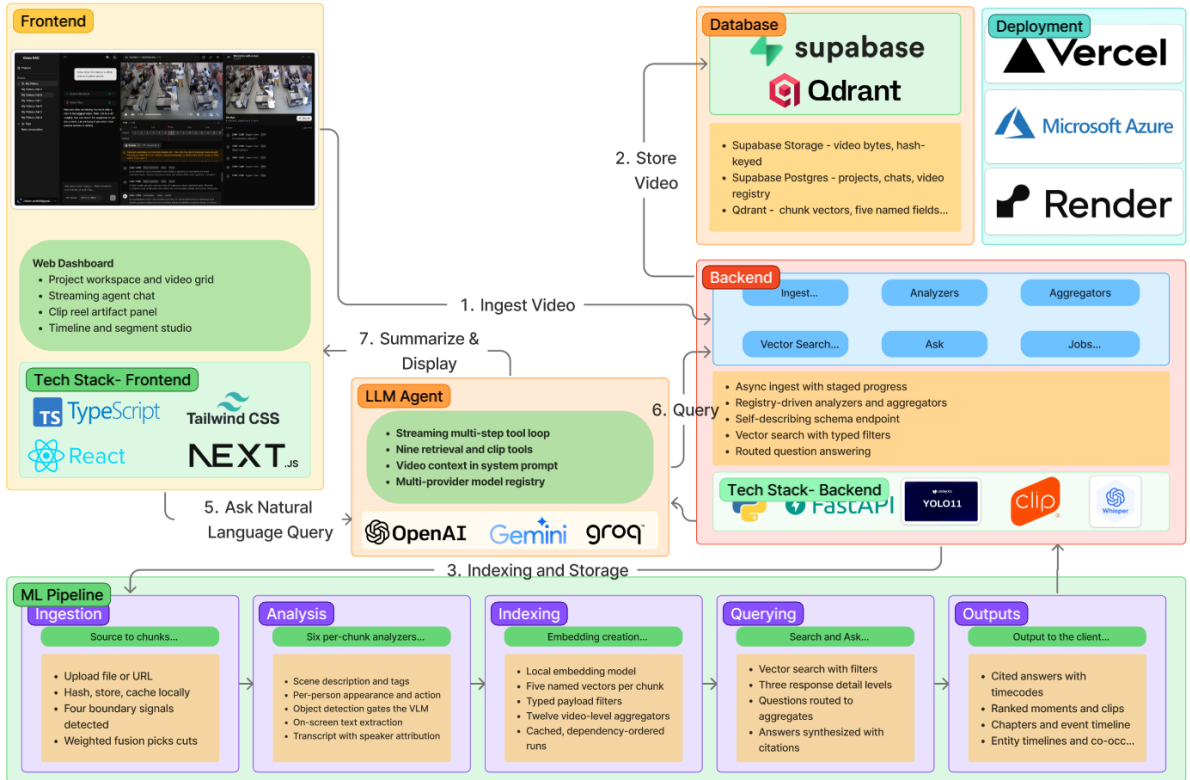


Figure 1: FalconVQA system architecture: client and agent tier, core analysis service, persistence, deployment targets, and the five-phase processing pipeline.

2.1 Partitioning the Recording

The recording is cut where its content changes. Four boundary detectors contribute evidence for a cut: a change of speaker, a period of silence, a hard scene cut, and a shift in the visual content of the frame. Their scores are fused under a weighting, and the strongest peaks of the

fused signal are taken as boundaries.

Where a signal produces no events at all, its weight is redistributed across the signals that did fire. This behaviour is essential rather than incidental: fixed-camera footage contains no hard cuts and frequently no speech, and without redistribution the entire recording is returned as a single chunk. Two further modes are provided for cases in which fusion is not wanted: a caller-supplied weighting, or fixed spans of N seconds with no detectors run at all.

The chunking configuration is stored alongside the resulting records, so that a single recording may be indexed both coarsely and finely at the same time, with a query resolving against whichever indexing is appropriate.

2.2 Analysing Each Chunk

The analyzers to be run are selected at upload time, because their costs differ by orders of magnitude. Six analyzers are provided: scene description, per-person description, object detection, on-screen text recognition, transcription, and speaker-attributed transcription. Transcription and speaker-attributed transcription are mutually exclusive, as the latter is a strict superset of the former.

The vision-language model is the dominant cost, so inexpensive detectors are run first and act as gates. If YOLO detects no object in a frame and EasyOCR finds no text, that frame is never submitted to the vision model. Frames are additionally deduplicated: a frame is retained only if it differs sufficiently from the last frame retained. The similarity threshold is adaptive with an absolute ceiling, a configuration arrived at by measurement: a fixed threshold retained a single frame on a static camera, and a purely relative threshold began selecting compression noise. Frames are read sequentially rather than by seeking, which was measured to be approximately $22\times$ faster.

2.3 Indexing and Aggregation

Each chunk is stored as a single point carrying five separately embedded vectors rather than one: the complete flattened record, the scene description, the people, the actions, and the objects. A query concerning a person's appearance is therefore matched against a short description of a person rather than competing against everything else recorded for that chunk. Embeddings are computed locally, and each point carries a typed payload against which structured filters are applied.

Twelve aggregators then run over the saved analyzer output, not over the video, in an order derived from their declared dependencies. They produce the video-level material: statistics, novelty, speaker statistics, summary, chapters, events, named entities, sentiment, linked person entities, linked object entities, entity timelines and co-occurrence. Because they consume stored records, re-running an aggregator incurs no re-analysis; results are cached and recomputed only on demand or when the analyzer set changes.

2.4 Search and Question Answering

Searches may be restricted by time, objects, tags, people and other typed filters, and return one of three levels of response detail, selected according to whether the consumer is a human reader or a token-metered agent.

Questions follow a different path. Rather than searching chunks and synthesising from whatever is returned, the question is routed to the video-level aggregates capable of answering it: the entity narratives for a question about a person, novelty for a question about anomalies, statistics for a question about activity levels.

In both cases the operator supplies a natural-language description and receives ranked moments carrying `mm:ss` timecodes. Selecting a result begins playback of the recording at that

point.

3 Requirements Analysis

3.1 Functional Requirements

Functional requirements are stated from the operator’s perspective: what a user of the system must be able to do with their footage. Requirements FR-1 to FR-12 are implemented across the client application and the core analysis service.

ID	Capability	The system shall allow the operator to...
FR-1	Account and workspace	Register, authenticate, and organise recordings into named projects, with each operator able to reach only their own projects and recordings.
FR-2	Adding footage	Add a recording to a project, either as an uploaded file or as a link, and select how thoroughly it is to be examined, trading processing cost against the depth of detail captured.
FR-3	Tracking progress	Observe that a recording is being processed, which stage it has reached and how far it has advanced, and continue working while processing proceeds.
FR-4	Searching	Locate moments by describing what occurred in natural language, without knowing when it occurred or what it was called.
FR-5	Narrowing results	Restrict a search to a period of the recording, to footage containing particular people, objects or spoken content, or to a chosen subset of recordings.
FR-6	Asking questions	Pose a question about one or more recordings and receive a direct answer accompanied by references to the moments from which it was derived.
FR-7	Reviewing results	Determine exactly when each result occurred and play the footage from that moment, individually or as a continuous reel of moments.
FR-8	Understanding a recording	Review a recording as a whole without watching the footage: its summary, chapters, notable events, the people and objects that appear and when, and any anomalies.
FR-9	Conversing about footage	Conduct a multi-turn conversation about a project’s recordings in which the assistant selects the appropriate retrieval operations, and have that conversation persist for later resumption.
FR-10	Inspecting the index	Examine what the system recorded for any individual chunk (scene description, on-screen text, people, objects and speech) alongside a timeline synchronised with playback.
FR-11	Re-examining footage	Re-examine a recording already held by the system at a different depth of analysis, without re-adding it and without discarding results already obtained.
FR-12	Programmatic access	Drive every retrieval capability over a documented HTTP API whose analyzers, filters and response shapes are discoverable from the API itself.

3.2 Non-Functional Requirements

Non-functional requirements are stated as service targets an operator would observe. The stated figures are design budgets to be confirmed by benchmarking, not measured results.

ID	Quality	Requirement
NFR-1	Responsiveness	A description search returns ranked moments within 2 seconds; a question is answered within 15 seconds.
NFR-2	Processing time	A recording becomes searchable within approximately three times its own duration, and the operator may continue working on other recordings while it is processed.
NFR-3	Transparency	Processing stage and progress remain visible throughout ingestion, refreshed at least every 5 seconds, so that a prolonged wait is never unexplained.
NFR-4	Playback latency	Selecting a result begins playback of the footage at that moment within 2 seconds, with no manual scrubbing.
NFR-5	Traceability	Every answer cites the moments from which it was derived, and the system states that the footage does not support an answer rather than producing one.
NFR-6	Robustness	Ordinary surveillance footage, meaning unedited, frequently silent and recorded from a fixed camera, is handled as the normal case rather than as a failure case.
NFR-7	Cost control	The operator selects how thoroughly each recording is examined, and reviewing results already obtained incurs no further processing cost.
NFR-8	Extensibility	A new analyzer or aggregator is added as a single module and one registry entry, with no modification to ingestion, storage or the API.
NFR-9	Data isolation	Recordings, projects and conversations are readable only by the account that owns them, enforced at the database and at the boundary through which the agent reaches the analysis service.
NFR-10	Idempotent ingestion	A recording is identified by the content of its bytes, so that re-submitting the same file is recognised as the same recording and is neither stored nor processed twice.

4 Creative Enhancements

The enhancements recorded below were, in most cases, adopted in response to a measured failure on real footage. Collectively they are what distinguishes the system from a conventional “partition the video and embed the captions” pipeline. Each is annotated with the requirement it primarily serves.

#	Enhancement	Rationale	Serves
E-1	Multi-signal chunking with weight redistribution	Four detectors contribute evidence for each cut, and a signal that produces no events cedes its weight to the signals that did. Without this, fixed-camera footage is returned as a single chunk, and a preset degenerates into a granularity dial: on single-signal footage the audio and video presets yielded 12 and 63 boundaries respectively from identical content.	NFR-6
E-2	Concurrent chunkings of one recording	The chunk configuration is stored with the records, so a recording may be indexed coarsely and finely simultaneously and a query resolves against whichever is appropriate.	FR-11
E-3	Five named vectors per chunk	Description, people, actions and objects are embedded separately alongside the combined record, so a query about a person’s appearance is matched against a short person description rather than competing with everything else recorded for that chunk.	FR-4
E-4	Inexpensive detectors used as gates	YOLO and EasyOCR determine whether a frame merits submission to the vision–language model. Frames containing nothing to see or read incur no model cost.	NFR-7
E-5	Detector labels withheld from the vision model	YOLO classified a thermally imaged tank as an “airplane” with 0.73 confidence. Shown the same bounding boxes without labels, the vision model reported a tank. A label rendered onto the image invites agreement with it.	NFR-5
E-6	Adaptive frame sampling	Frames are retained according to their dissimilarity from the last frame retained. A fixed threshold of 0.95 retained one frame on a static camera, whose consecutive similarity median was 0.9955; a purely relative threshold selected compression noise. Both were measured.	NFR-2
E-7	Questions routed to aggregates	A question is directed to the video-level output capable of answering it (entities for a person, novelty for anomalies, statistics for activity levels) rather than searching segments and synthesising from whatever is returned.	FR-6
E-8	Constrained entity linking	People are clustered on appearance under hard constraints, chief among them that two people visible in the same chunk cannot be the same person, and this happens before an LLM writes any narrative. Asked to match people directly, an LLM merges anyone in dark clothing. The signature is clothing alone: blending in the appearance field dragged same-person scores to 0.69–0.87, because it drifts and occasionally carries meta-commentary that the embedding treats as content.	FR-8
E-9	Object linking with fixture suppression	Object entities reuse the same constrained clusterer with two additional filters: static fixtures such as counters and shelves are counted but not tracked, since the observation that the counter was present throughout is true of every frame and informs no search; and person-like detections are handed to the people aggregator, which has clothing with which to identify them.	FR-8
E-10	Selectable response detail	The same search returns a few hundred tokens or tens of thousands. Five full hits were measured at approximately 19,000 tokens against approximately 440 at minimal . A human reader wants the whole record; an agent pays per token.	NFR-7
E-11	Self-describing API	A single endpoint returns every analyzer, vector field, filter and aggregator the installation supports, so a client or an LLM agent discovers what may be asked instead of being hard-coded against it. The upload dialog builds its analyzer list from this endpoint, so that adding an analyzer remains a change to one module and one registry line.	FR-12, NFR-8
E-12	Content-addressed recordings	A recording is identified by the hash of its bytes, so the same file submitted twice is recognised as the same recording and is neither processed nor stored again. No local path is ever persisted: the cached file is re-derived from the content hash	NFR-10

5 Detailed Design of the Processing Pipeline

This section specifies each of the four pipeline stages in turn: the models employed, the parameters governing them, and the output each stage produces for the next. Every numeric value given is the value in the implementation.

5.1 Stage 1: Chunking

Purpose and inputs

The stage receives a decoded video file and produces an ordered list of non-overlapping time spans covering the recording. Audio is decoded once with PyAV into a mono waveform, from which the duration is derived; the video stream is read separately by the visual detectors.

Boundary detectors

Four detectors run once per recording. Each returns a list of (time, strength) events, where the strength lies in $[0, 1]$.

Signal	Model / algorithm	Event derivation	Strength
Speaker change	pyannote/speaker-diarization-community-1 (gated, requires a Hugging Face token)	The waveform is diarized into speaker turns; a boundary is emitted at the midpoint between two consecutive turns whose speaker label differs.	Constant 1.0. A detected speaker change is a binary, high-confidence event.
Silence	Silero VAD v6	Speech timestamps are returned for the whole track; a boundary is emitted at the midpoint of every gap between speech segments longer than 0.05 s, including the trailing gap.	$\min(1, g/2.0)$ for a gap of g seconds, saturating at two seconds.
Hard cut	PySceneDetect ContentDetector ; purely algorithmic frame differencing, no model weights	A boundary is emitted at the start of every scene after the first.	Constant 1.0. The detector applies its own threshold, so any report is confident.
Semantic drift	OpenCLIP ViT-B/32, OpenAI weights, frames sampled at 1 fps	Consecutive sampled frames are embedded and compared; a boundary is emitted at each midpoint with raw strength $1 - \cos(e_i, e_{i+1})$.	Rescaled by the clip’s own largest jump, so the strongest semantic change is placed on equal footing with a speaker change or a cut.

The rescaling applied to the semantic signal is necessary because CLIP similarity drifts continuously and rarely approaches zero even at a genuine scene change; without it the signal would be permanently diluted relative to the sparse, unit-strength detectors.

Frames for the semantic detector are read by walking the container forward with `grab()/retrieve()` rather than seeking to each timestamp. Per-frame seeking forces the H.264 decoder to restart from a keyframe and was measured at approximately $22\times$ the cost per frame.

Fusion

Weights are first renormalised over the signals that produced at least one event, so that a preset expresses which signals matter rather than acting as a granularity dial (Section 4, E-1). Each event then contributes a Gaussian bump to a score curve evaluated on a grid of resolution 0.1 s over the recording:

$$S(t) = \sum_{s \in S} \sum_{(t_i, a_i) \in E_s} w_s a_i \exp\left(-\frac{(t - t_i)^2}{2\sigma^2}\right), \quad \sigma = 1.0 \text{ s}$$

Here S is the set of signals, E_s the events emitted by signal s , w_s its renormalised weight, a_i the strength of event i , and σ the width of the Gaussian kernel.

Boundaries agreed upon by several detectors therefore reinforce one another. Peaks are selected greedily in descending order of $S(t)$, subject to a score floor of 0.12 and a minimum separation of 2.0 s between accepted boundaries, which prevents runs of adjacent micro-chunks.

Preset	speaker	silence	cut	semantic	Intended footage
audio	0.35	0.35	0.15	0.15	Dialogue-led recordings
video	0.15	0.15	0.35	0.35	Edited or visually varied footage
audio_video	0.25	0.25	0.25	0.25	Default; all four signals equally

Modes and post-processing

Three mutually exclusive modes are exposed. In **preset** mode a named weighting is used; in **weights** mode the caller supplies its own, validated against the four signal names and required to be non-zero in at least one; in **interval** mode the recording is cut into fixed spans of N seconds and no detector runs at all.

For the two weighted modes, a span of length d longer than the maximum duration $D_{\max}$ is divided into $\lceil d/D_{\max} \rceil$ equal parts. Equal division is used rather than truncation, so that no negligible trailing fragment is produced. A span shorter than the minimum duration is merged forward into its successor, with a short final span folded back into the preceding one. The API defaults are 5 s and 20 s. Duration bounds are deliberately not applied in **interval** mode, since the caller requested exact spans.

Output

An ordered list of (start, end) spans, together with a chunk configuration key that identifies the scheme that produced them, for example **audio_video:5-20** or **interval:10**. Custom weightings are hashed into the key so that two different weightings sharing the same duration bounds cannot collide. This key is carried on every downstream vector, which is what permits two chunkings of one recording to coexist.

5.2 Stage 2: Analysis

Purpose and inputs

Each selected analyzer receives the chunk spans and a video context that provides frames and audio on demand. Every analyzer owns its own frame sampling, its own prompt and its own output schema, and is registered by identifier; nothing in ingestion, storage or the API names a specific analyzer.

Common sampling strategy

The frame-selecting analyzers share a three-part strategy. Candidate frames are drawn at a low fixed rate; near-duplicates are removed by comparing OpenCLIP embeddings against the last frame retained; and the survivors are thinned to a per-analyzer maximum. The scene analyzer derives its similarity threshold adaptively, as the 40th percentile of consecutive-frame similarity within the chunk, capped at 0.98. The cap is load-bearing: on a static chunk the percentile lands on the median, measured at 0.9955, and distinct frames are then mined out of compression noise. The detector-gated analyzers instead use a fixed similarity threshold together with a layout-change test, because a person or object entering or leaving the frame is a change the whole-image comparison does not register.

All frames selected for a chunk are sent in a single request, so that the model sees the chunk as a continuous sequence and can describe movement across it rather than judging each frame in isolation. Sampling is serial, because the CLIP model is shared on one GPU; the API calls are I/O-bound and run across six worker threads.

Analyzers, models and outputs

The vision-language model is invoked with OpenAI strict structured output, so every required property is present and no additional properties are returned; downstream stages therefore never parse prose.

Two design rules apply across the table. First, whole-track passes are performed once and memoised: a per-chunk diarization pass would label the same person `SPEAKER_00` in one chunk and `SPEAKER_01` in the next, and per-segment language detection produced fluent German from an English clip. Second, `transcript` and `diarization` are mutually exclusive and the API enforces this, because their embedded vectors were measured cosine-identical at 1.0000: diarization is a strict superset.

Gating and cost

Where an analyzer’s gate finds nothing, meaning no person above the area threshold, no object detection and no text region, the chunk yields no output and no API call is made. A frame that survives is downsampled to the analyzer’s width and JPEG-encoded before transmission, since vision tokens are priced by resolution.

Analyzer	Models and gating	Structured output
default_video	Candidates at 2 fps; OpenCLIP ViT-B/32 deduplication (adaptive threshold, cap 0.98); 4–8 frames retained, thinned against evenly spaced points in time; VLM gpt-5.4-mini at 768 px.	Schema <code>scene_description</code> : <code>description</code> (the prose users search against), <code>setting</code> , <code>people[]</code> , <code>objects[]</code> , <code>actions[]</code> , <code>tags[]</code> . The timestamps of the frames used are recorded alongside for traceability.
people	YOLO11n gate at 1 fps, confidence 0.35; person boxes below 900 px ² discarded; identifiers assigned by overlap tracking across every candidate frame, then read off for the frames actually sent; up to 6 frames annotated with numbered yellow boxes; VLM at 1280 px, <code>detail=high</code> .	Schema <code>people_reading</code> : per person <code>box_id</code> , <code>appearance</code> , <code>clothing</code> , <code>role</code> , <code>action</code> , plus a <code>summary</code> . Each person is post-processed with <code>locations</code> , holding the timestamp and box in every frame they appear in, and a <code>people_count</code> is recorded.
object_detection	Shares the same YOLO11n pass rather than repeating it; frames annotated with cyan boxes carrying no labels; up to 6 frames; VLM at 1280 px, <code>detail=high</code> .	Schema <code>object_reading</code> : <code>detections[]</code> of <code>object</code> , <code>description</code> (colour, material, size, condition) and <code>context</code> (what it is being used for, and by whom), plus a <code>summary</code> . Plain object names are extracted into a filterable list; the detector’s own labels are retained only for debugging.
ocr	EasyOCR detection stage only, since its recognition stage fragments lines and reads low-contrast overlays poorly; frames annotated with red boxes; similarity threshold 0.985 plus a text-layout change test; VLM at 1600 px, <code>detail=high</code> .	Schema <code>ocr_reading</code> : <code>texts[]</code> of <code>text</code> (verbatim, with case and punctuation) and <code>context</code> (what the text is and where it appears), plus a <code>summary</code> .
transcript	faster-whisper (<code>tiny</code>); language detected once over the whole track and pinned for every chunk.	One plain-text transcription per chunk.
diarization	pyannote over the whole track once, memoised; faster-whisper segments per chunk; speaker assigned by largest temporal overlap between a Whisper segment and a pyannote turn.	<code>turns[]</code> of <code>speaker</code> , <code>start</code> , <code>end</code> , <code>text</code> with consecutive same-speaker segments merged, plus a rendered <code>text</code> , a speaker-free <code>plain</code> form, and the <code>speakers</code> present.

5.3 Stage 3: Indexing and Aggregation

Embedding

Every analyzer implements `render_fields`, which flattens its output into a map of named vector space to text. Five spaces are defined: `combined`, `description`, `people`, `actions` and `objects`, of which `combined` must always be present. A space is omitted rather than emitted empty, so a chunk in which the model saw nobody is simply not a candidate in a people-scoped search.

Text is embedded locally with `BAAI/bge-small-en-v1.5` through sentence-transformers, producing normalised vectors compared by cosine distance. The model family was trained with an instruction prefix on the query side only, so queries are prefixed with “Represent this sentence for searching relevant passages:” and documents are not. Applying the prefix to both, or to neither, measurably degrades recall. All fields of all chunks are embedded in one batched GPU pass and then scattered back to their (chunk, field) slots.

Vector store

One Qdrant point is written per chunk per analyzer, carrying all five named vectors. Because the point is the unit, a limit of ten returns ten chunks, the payload is stored once, and a filter is applied once. The point identifier is a deterministic UUID derived from `sha1(video_id | analyzer_id | start | end)` keyed on the time span rather than a positional index. Chunk 12 of one chunking run is a different moment from chunk 12 of another, so a positional identifier would silently rebind a vector to the wrong timeframe. Re-ingesting the same chunk therefore upserts rather than duplicating.

The payload carries everything required to cite and render a moment without joining back to disk: the video identifier and playback URL, the analyzer, the chunk configuration and identifier, the time span, the combined text and description, the setting, the people, objects, actions and tags, the speakers and their turns, the read texts, the object detections, the full per-person records, and the person count. Twelve payload fields are declared as indexes, and twelve named filters are defined in a single filter specification: video, chunk and analyzer identifiers, chunk configuration, objects, tags, speakers, people, minimum and maximum person count, and time bounds. Filters are validated against that specification and an unknown name raises an error with the nearest suggestion, because a silently ignored filter returns plausible but wrong results.

Aggregators

Twelve aggregators run over the saved analyzer records, in an order derived from their declared dependencies. A dependency naming an analyzer is a requirement on the recording, and an aggregator whose analyzer is absent is dropped rather than run against nothing. Five of the twelve incur API calls; the remainder are free, which is what makes re-running them on demand practical.

Entity linking in detail

Identity is decided by embedding and clustering, not by asking a model to match people. Measured on this footage, provably different people, meaning people co-visible within one chunk and so unable to be the same, reached 0.941 similarity, while known-same pairs sat between 0.889 and 0.915. Those bands overlap, so no threshold alone is sufficient and the constraints carry the weight. The procedure is:

1. **Signature construction.** The signature is the `clothing` field alone. Blending in `appearance` dragged same-person scores down to 0.69–0.87, because that field drifts between chunks and occasionally carries meta-commentary that the embedding treats as content.

Aggregator	Depends on	Model	Output
stats	None	Arithmetic only	Duration, chunk count, person count series with minimum, maximum, mean, busiest and quietest spans; object frequencies; word count, spoken seconds and speech ratio.
novelty	None	Reuses stored vectors	Cosine distance of each chunk's combined vector from the normalised centroid of the recording, ranked; outliers are those beyond two standard deviations, so "unusual" scales with how varied the recording itself is.
speaker_stats	diarization	Arithmetic only	Per speaker: seconds, turns, words, share of speech, longest turn and chunks. Plus hand-over count, dominant speaker and a turn timeline.
summary	None	LLM	A hierarchy built by repeated halving. Depth is derived from chunk count, taking enough halvings for a leaf to cover at most five chunks and bounded to between three and six tiers, so that leaf granularity does not vary with recording length. Leaves are summarised from chunk text and every tier above merges its children, making the whole-recording summary a reduction rather than one enormous prompt. Emits summary , key_points[] , topics[] , the full tier hierarchy, and the finest tier as sections for retrieval.
chapters	None	LLM	Runs of consecutive chunks grouped into named sections with a title and one-sentence summary. Chapters must be consecutive, non-overlapping and exhaustive; any chapter referencing a chunk identifier that does not exist is discarded.
events	None	LLM	Discrete, chunk-anchored occurrences: event , chunk_id , actor , category , resolved to real time spans and counted by category. Ongoing background activity is explicitly excluded.
ner	None	GLiNER urchade/ gliner_small-v2.1, zero-shot, threshold 0.5	Named entities over descriptions, on-screen text and speech across ten configurable labels. Pronouns and bare determiners are excluded by a stop list, as they are grammatically entities but identify nobody and would dominate a mention count. Emits entities with mention counts, scores and chunk identifiers, grouped by label.
sentiment	diarization (falls back to transcript)	cardiffnlp/ twitter-roberta- base-sentiment-latest	A per-turn timeline, per-speaker distribution with dominant label and negative share, and an overall distribution. Run on speech only, and reported as sentiment of language observed rather than emotion inferred.
entities	people	BGE embeddings and constrained clustering, then LLM narratives	Person observations linked into individuals. See below.
object_entities	object_detection	The same clusterer, plus lexical filters ¹⁵	Object sightings linked into individual objects. Static fixtures are counted but not tracked, and person-like detections are handed to the people aggregator.
entity_timeline	entities	None	Per person: first and last sighting, total span, observed presence summed over sight-

2. **Within-chunk merge.** Observations in the same chunk above a similarity of 0.95, which is above the measured co-visible ceiling, are merged first. The analyzer’s own tracker loses a person who moves between sampled frames, and without this step the cannot-link constraint would permanently forbid reuniting them.
3. **Distinctness gate.** Each observation’s mean similarity to all others is computed. Anything at or above 0.80 describes half the recording and identifies nobody, so its owner is left unlinked rather than merged into whichever cluster it drifted closest to.
4. **Constrained agglomeration.** Pairs above 0.88, drawn from different chunks and both distinctive, are merged greedily in descending order of similarity. A merge is rejected if the two clusters share any chunk, since that would place one person in two places at once.
5. **Narration.** Only then is an LLM used, and only for people seen more than once, capped at twelve narratives per recording. It produces a one-line recognition description, an ordered account of what the person did, and an apparent role.

Object linking reuses the same routine on a source where identity is inherently weaker, since two shopping carts genuinely look alike where two people rarely wear the same clothes, so the distinctness gate does most of the work.

5.4 Stage 4: Retrieval and Question Answering

Search

A search embeds the query with the query-side prefix, selects one of the five named vector spaces, applies the validated filter set, and returns ranked chunks with their time spans and payloads. Two parameters deserve note.

Score threshold. Cosine similarity always ranks something, so a query for content a recording does not contain still returns its nearest neighbours. A score threshold is what makes “no match” expressible. On this data, content actually present scored ≥ 0.665 and absent content ≤ 0.524 , so a threshold of approximately 0.55–0.60 separates them; the value is specific to the embedding model and is retuned if the model changes.

Detail level. Three levels are offered: `minimal`, `standard` and `full`. Five full hits were measured at approximately 19,000 tokens against approximately 440 at `minimal`. The interface requests the whole record; an agent, which pays per token, requests less.

Question routing

A question is not answered by searching chunks and hoping. Each aggregate carries a hint, which is a short description of what that aggregate is good for, written in the vocabulary a question would use. The hints are embedded and compared against the question, and the aggregates standing at least one standard deviation above the mean score for that question are selected, capped at three. If nothing stands out, the single best-matching aggregate is used, so a broad question is answered from the strongest match rather than from nothing.

Relative selection is used because absolute thresholds do not survive contact with real questions: the score distribution shifts per question, with means between 0.47 and 0.56 measured here, so any single cutoff either admits everything or nothing. The ranking, however, is reliable. Because routing is by embedded description rather than by keyword, a new aggregator joins by adding one line of hint text, and unanticipated phrasings degrade gracefully.

Aggregate	Questions it is routed
entities	What a particular person did; someone identified by clothing or appearance; following one individual; how long a person stayed.
novelty	Which segment is unusual, anomalous or out of the ordinary; what stands out.
stats	How many people were present; how busy or crowded; the busiest or quietest moment; counts and totals.
events	What happened and when; arrivals, departures and transactions in time order.
chapters	The structure of the footage; what sections it has; what the recording is about overall.
speaker_stats	Who talked most; how many speakers; speaking time, turns and interruptions.
sentiment	The tone of what was said aloud; whether spoken language was positive, negative or neutral.
ner	Named things mentioned: brands, organisations, places, names, dates and amounts.
object_entities	A particular object followed through the footage; what an item was used for over time.
cooccurrence	Which people appeared together; who was accompanied by whom.

Context assembly and synthesis

The routed aggregates are rendered into labelled blocks, each carrying explicit timecodes. The overall summary is always included, together with the four summary sections closest to the question and, where entities were routed, the four people whose descriptions best match it. Matching chunks from vector search are appended last. The response records which aggregates were routed to and how much of each was used, so an answer can be traced to its sources.

The synthesis prompt constrains the model to the supplied material, requires every claim to carry a [video_id start-end] citation drawn from the timecodes given, and directs it to state plainly that the material does not support an answer rather than guessing (NFR-5).

Output to the client

The stage returns cited answers with timecodes, ranked moments with playable time ranges and playback URLs, and from the aggregates chapters, the event timeline, entity timelines, co-occurrence, statistics, novelty and sentiment. A moment is delivered as a range over a recording rather than as a rendered file, so several moments drawn from one recording share a single media load in the client.

6 Architecture and Technology Stack

6.1 Design Methodology

The system was developed iteratively, one pipeline stage at a time, each stage operating end to end before the next was begun. Two principles governed most of the design decisions:

- **Measurement precedes commitment.** Every threshold and sampling choice in the pipeline was established by executing it against real footage and comparing outcomes, rather than by selecting a plausible value. Several of the enhancements listed in Section 4 exist because the obvious choice was adopted first and observed to fail.
- **Extension by registration, not by modification.** Analyzers and aggregators are declared in a single registry and discovered from it. No part of ingestion, storage or the API names a specific analyzer, so introducing one does not require any of them to be edited.

Verification was performed against a running server rather than by code inspection, because a number of defects (stale processes, broken answer synthesis, incorrect media paths) were observable only over HTTP.

6.2 Architectural Overview

The system is a three-tier application. A Next.js client provides the operator interface and hosts an LLM agent that translates conversational requests into retrieval calls. A FastAPI service owns the entire video pipeline and exposes it as a self-describing HTTP API. Persistence is divided across object storage for media, a relational database for application state, and a vector database for retrieval.

Composition within the analysis service is deliberately in-process: aggregators consume analyzer output directly rather than over HTTP, and HTTP is reserved for the external boundary. Analyzers and aggregators are held in registries, which is what makes NFR-8 achievable, because the ingest path, the store and the API iterate over a registry and never name a specific analyzer.

The analysis service has no concept of users, projects or row-level security: any caller able to reach it can read every recording it holds. Multi-tenancy is therefore enforced entirely in the client tier, by row-level security on the database and by a single scoping function through which every agent tool resolves the recording identifiers it is permitted to pass downward (NFR-9).

Layer	Responsibility
Client	Marketing site and documentation, authentication, project workspace, video upload and status, agent chat, clip artifact panel, video studio timeline, and the per-recording detail view.
Agent layer	Multi-step streaming tool loop. Converts a natural-language request into search, question, inspection and clip operations, and streams results and artifacts back to the client.
API route layer	The client's own server routes: project scoping, row-level security, job reconciliation, and proxying to the analysis service.
Core service	Ingest orchestration, chunking, analyzers, aggregators, vector search, question answering, clip and poster export, job tracking.
Storage boundary	The single module that knows both a storage URL and a local path; everything above it speaks URLs, everything below it receives paths.
Persistence	Object storage for video bytes, relational store for application state, vector store for chunk embeddings, local cache for decoded media and model weights.

6.3 Module Inventory

Tier	Module	Responsibility
Core	<code>chunk.py</code> , <code>chunking/</code>	The three chunking modes and the weighted fusion of boundary evidence.
Core	<code>boundaries/</code>	The four detectors: speaker change, silence, hard cut, semantic drift.
Core	<code>analyzers/</code>	Six per-chunk passes, each owning its frame sampling, prompt and output shape.
Core	<code>aggregators/</code>	Twelve video-level passes over saved records, ordered by declared dependencies.
Core	<code>extractors/</code>	Shared primitives: frame readers and samplers, audio decoding, vision–language calls, detector wrappers.
Core	<code>vectordb/</code>	Local embedding, the Qdrant point schema with five named vectors, and record rendering.
Core	<code>api/</code>	<code>core.py</code> holds the logic; <code>app.py</code> is a thin HTTP layer; <code>ask.py</code> performs routed question answering; <code>jobs.py</code> tracks background ingestion.
Core	<code>storage.py</code> , <code>paths.py</code>	The storage boundary and every filesystem path, each overridable by environment variable.
Core	<code>export.py</code> , <code>poster.py</code>	Per-chunk MP4 export by a single decode pass, and one poster frame per recording.
Client	<code>app/projects/</code>	Project list, workspace, upload dialog, media grid, and the per-recording detail view.
Client	<code>app/agent/</code>	Chat surface, message and tool renderers, artifact panel, clip reel, and the video studio timeline.
Client	<code>app/api/agent/tools/</code>	The nine agent tools and the project scoping function that binds them.
Client	<code>app/api/videos/</code>	Ingestion, status, aggregates, timeline, details and re-index routes.
Client	<code>lib/core/</code>	Typed client for the analysis service, job reconciliation, chunking configuration and formatting helpers.
Client	<code>lib/supabase/</code>	Browser, server and admin clients, session middleware, and the schema migrations.
Client	<code>content/docs/</code>	The MDX documentation site: concepts, workspace guides and the full API reference.

6.4 Ingestion Lifecycle

Ingestion is asynchronous. The service accepts a file or a link, returns immediately with a job identifier, and processes the recording on a background thread through the stages specified in Section 5. The job reports its stage as fetching, chunking, analyzing, indexing, aggregating or complete, and the client polls it, reconciling the result into the recording’s row. Because jobs are held in memory, a restart mid-ingest is detected by the client, which either recovers the recording from its persisted records or fails the row explicitly so that re-indexing becomes the route forward.

6.5 Persistence

State is divided across four stores: a relational database for application state, object storage for media, a document store on disk for analyzer and aggregator output, and a vector store for

retrieval. The division, the schema of each store and the identity that joins them are specified in Section 7.

6.6 Technology Stack

Concern	Technology
Frontend	Next.js 16, React 19, TypeScript 5, Tailwind CSS v4, shadcn/ui on Radix primitives
Agent & models	Vercel AI SDK v6; OpenAI, Google, Groq, Cerebras and xAI providers behind one model registry
Client state	Zustand, TanStack Query
Playback	video.js, hls.js
Documentation	Fumadocs over MDX, served from the same application
Auth & tenancy	Supabase Auth with Postgres row-level security
Backend	Python 3.13, FastAPI, Uvicorn
ML runtime	PyTorch 2.11 (CUDA 13), Transformers
Boundary detection	pyannote.audio (speaker change), Silero VAD (silence), PySceneDetect (hard cuts), OpenCLIP ViT-B/32 (semantic drift)
Analysis models	Vision-language model for scene, people, object and text description; faster-whisper for transcription; YOLO11 and EasyOCR as detection gates
Aggregation models	GLiNER for zero-shot named-entity recognition, a transformer classifier for sentiment, an LLM for narrative aggregates
Embeddings	BGE-small-en-v1.5 via sentence-transformers, executed locally
Vector store	Qdrant, one point per chunk with five named vectors
Media I/O	OpenCV, PyAV, soundfile, Pillow
Application data	Supabase: Postgres, Auth and Storage
Deployment	Vercel (client), CUDA-capable host (core service), Supabase Cloud, Qdrant

Justification of Principal Choices

- **Python for the analysis service.** Every model the system depends upon (diarization, CLIP, Whisper, YOLO, the embedding model) is distributed as a Python library first. No other runtime would have permitted this number of models to be assembled in the time available.
- **FastAPI.** Asynchronous request handling for long-running ingest jobs, and generated API documentation, which is material when an LLM agent is one of the clients.
- **Qdrant.** Named vectors are the determining factor. Storing several vectors per chunk under one point is a first-class capability there, and it is what enhancement E-3 rests upon. It also runs embedded, so development requires no separate server.
- **A local embedding model.** Embedding is performed on every chunk of every recording. Paying an API per embedding would make re-indexing expensive; a small local model makes it free.
- **Next.js and the Vercel AI SDK.** Streaming responses, tool calling and multi-provider model selection are provided, so the agent interface reduced to defining tools rather than building a chat runtime.

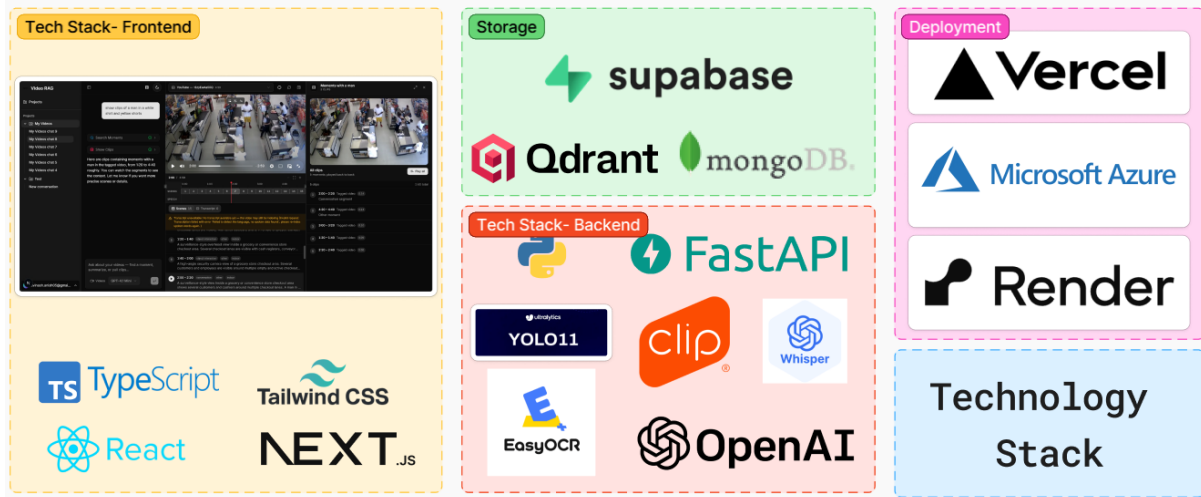


Figure 2: Technology stack across the frontend, backend, storage and deployment tiers.

- **Supabase.** Postgres, authentication, row-level security and file storage from a single service, which together constitute the whole of the non-video state.

6.7 System Requirements

Component	Requirement
GPU	CUDA-capable NVIDIA GPU, 8 GB VRAM minimum (developed on an RTX 4060). Required for diarization, CLIP, Whisper, YOLO and the embedding model.
Runtime	Python 3.13 for the analysis service; Node.js 20 or later for the client.
Memory / storage	16 GB system RAM; disk sized for the media cache, which holds one decoded copy of each ingested recording keyed by content hash.
External services	Object storage and Postgres (Supabase project); an LLM provider API key for vision analysis and answer synthesis; a Hugging Face token for the gated diarization model.
Network	Outbound HTTPS for model downloads, object storage and LLM calls. Ingestion executes as a background job, so link latency does not block the client.

6.8 Known Limitations

The following limitations are known and accepted for the current phase:

- **Entity linking is confined to a single recording.** The clustering step generalises to a camera network unchanged, but the available footage shares no people, so it could not be validated.
- **Within-chunk person tracking fragments.** Bounding-box association at 1 fps loses a person who moves between sampled frames, so a box identifier is a referent within a chunk rather than an identity claim.
- **Payload indexes are inert in embedded mode.** Filtering is correct but scans. Running Qdrant as a server resolves this with no code change.
- **Jobs are held in memory.** A restart discards job history; records and vectors survive on disk, and the client detects and recovers from the lost job.

- **Chapters may collapse to one** on unbroken single-location footage. This is arguably correct but of little use in the interface.

6.9 Scope for Future Enhancements

The design reserves room in the areas expected to be extended next:

- **Cross-recording person linking.** People are already linked within a recording; performing the same clustering step over a wider set extends this across an entire camera network once suitable footage is available.
- **Live camera input.** Ingestion presently accepts a recording. The same pipeline applies to a rolling window from a live feed, since chunking operates on segments rather than whole files.
- **Visual re-identification.** Where clothing descriptions are too generic to separate two people, the stored bounding boxes provide the crops required to compare them visually.
- **Additional analyzers.** Any pass that reads a chunk and returns structured output, such as licence plates, pose estimation or crowd density, may be added as a single module with no change elsewhere.
- **Agent access over MCP.** The API already describes itself; exposing it as a tool server would permit an external assistant to use it directly.

7 Database Design

7.1 Storage Strategy

Persistence is divided across four stores rather than one, because the kinds of state the system holds have incompatible access patterns and forcing them into a single store would compromise all of them.

Store	Technology	Contents	Unit
Application database	Supabase Postgres	Accounts, projects, the recording registry, conversations and their messages. Normalised, transactional, and subject to row-level security.	Row
Object storage	Supabase Storage, two buckets	Video bytes as uploaded, the canonical copy the pipeline reads, and one poster frame per recording.	Object
Analysis record store	JSON documents on local disk	Everything the analyzers and aggregators produced for one recording, nested under its chunks.	Document
Vector store	Qdrant, embedded	One point per chunk per analyzer, carrying five embeddings and a filterable payload.	Point

Analysis output is held as documents rather than as relations because its schema is defined by the analyzer registry, not by the database: each analyzer declares its own output shape, and a chunk carries one key per analyzer that ran on it. Modelling that relationally would make adding an analyzer a schema migration, which contradicts NFR-8. Application state is relational because it needs foreign keys, cascading deletes and per-row access control. Retrieval is separated again because approximate nearest-neighbour search over several named vector spaces is not something either of the other two can serve.

7.2 Application Database

Figure 3 gives the logical structure, and Figure 4 the physical schema as it stands in Postgres, with every column, its type and whether it is nullable.

Relation	Columns and notes
projects	id, user_id, name, description, timestamps. The grouping every other relation hangs from.
video_core	The application's view of a recording analysed by the core service, and the only place the two halves of the system meet. Identity and ownership (id, project_id, user_id, title); source and media (source_type, storage_path, source_url, core_video_id, playback_url, poster_url, duration, size_bytes); processing state (status, job_id, stage, progress, error); and what was asked for and produced (ingest_config, analyzers, aggregates, chunk_config, chunk_count).
conversations	id, user_id, project_id, title, lastContext, timestamps. A conversation is optionally attached to a project, and the attachment is what scopes the agent's tools.
messages	id, conversation_id, role, parts, metadata, created_at. parts is jsonb because a turn is a sequence of heterogeneous parts (text, tool calls, tool results and artifacts) whose shapes are set by the agent runtime.
videos	Superseded. The relation predates the current analysis service and is no longer read; video_core replaced it additively so that no migration had to drop data.

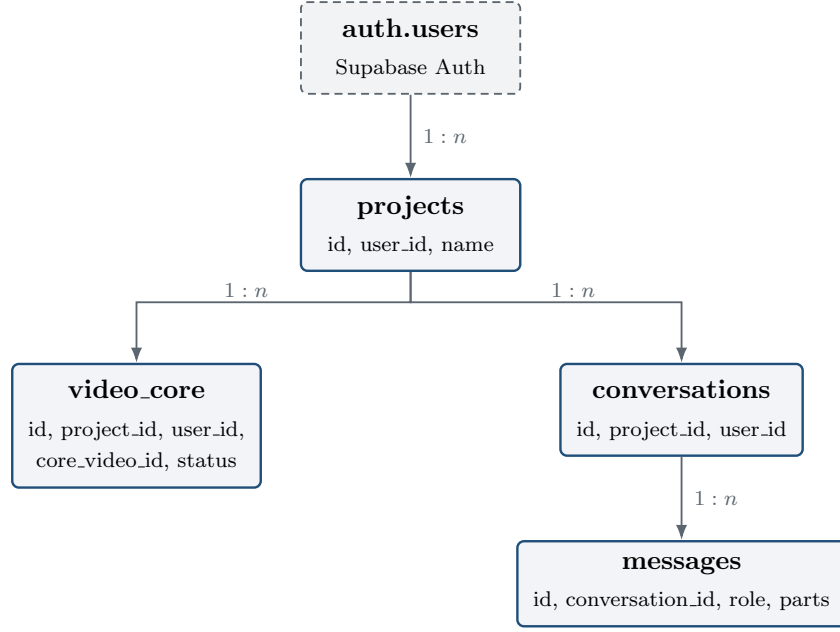


Figure 3: Application database. Every relationship cascades on delete, so removing a project removes its recordings, conversations and messages with it.

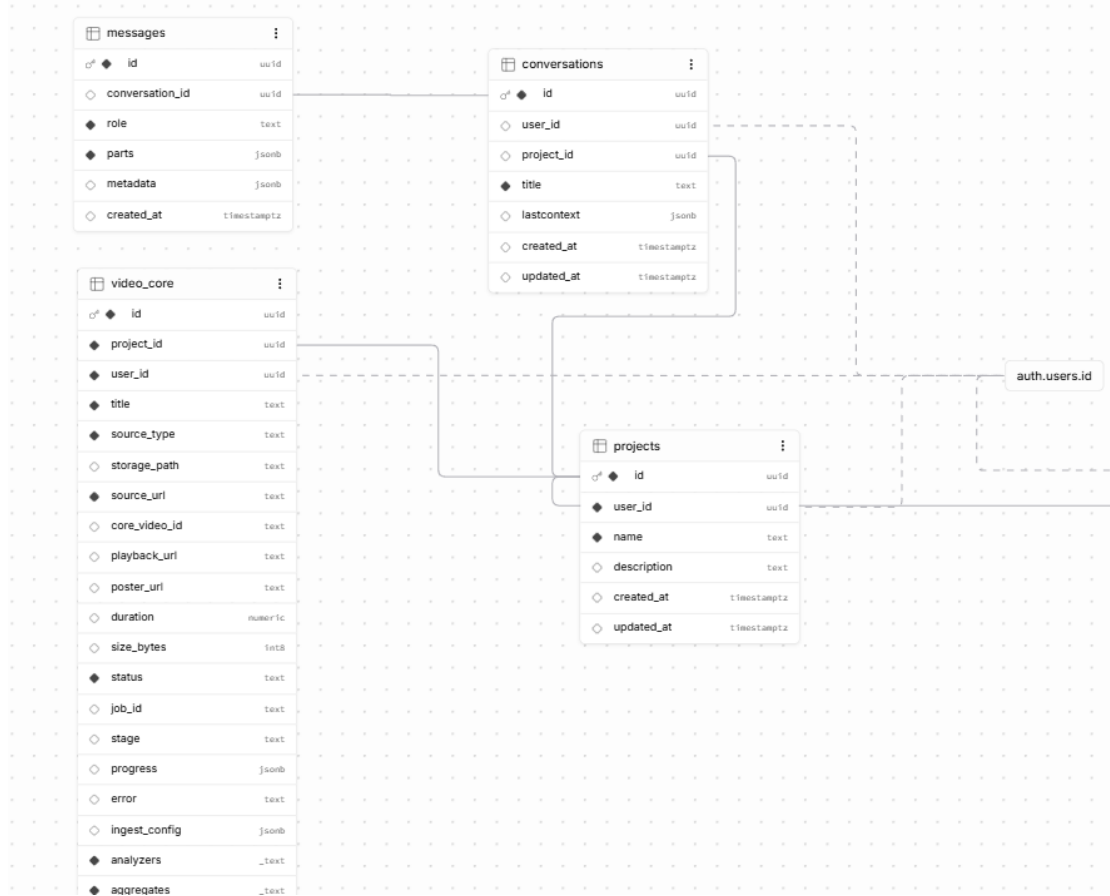


Figure 4: Application database as deployed: the four live relations with their columns, types and nullability, and their foreign keys into `auth.users`. Filled markers denote non-nullable columns.

Three decisions in `video_core` are load-bearing. Two URLs are kept rather than one, because the system spans two Supabase projects: this one holds the row and the object the browser uploaded, while the analysis service’s own project holds the canonical copy that plays. Uniqueness is per project and never global, since `core_video_id` is a content hash and the same file added to two projects yields the same value; the unique index is declared over (`project_id`, `core_video_id`) and is partial, restricted to rows where the column is non-null, because the identifier does not exist until the bytes have been downloaded. And the produced analyzers are stored as an array rather than as a document, because both the interface and the agent evaluate “does this recording have people analysis?” as a predicate, which an array answers directly. The requested configuration is `jsonb` by contrast, because it is replayed verbatim on re-index rather than queried.

- **Indexes.** `video_core` is indexed on (`project_id`, `created_at` DESC) for the workspace listing, and partially on `job_id` where it is non-null, which is the lookup reconciliation performs. `conversations` is indexed on `project_id` and `messages` on (`conversation_id`, `created_at`), the order a conversation is replayed in.
- **Integrity.** Every foreign key cascades on delete, so a removed account or project leaves nothing orphaned. A shared trigger function maintains `updated_at` on each relation that carries it.
- **Access control.** Row-level security is enabled on every relation. `projects`, `conversations` and `video_core` test `auth.uid() = user_id` directly. `messages` carries no owner column, so its policy tests ownership of the parent conversation through an EXISTS subquery. This is the database half of NFR-9; the other half is the project-scoping function through which the agent’s tools resolve recording identifiers.

7.3 Object Storage

Two public buckets are used, one in each Supabase project. `project-assets` in the application project holds what the browser uploads, laid out per user and project, with public read, insert restricted to authenticated accounts and delete restricted to the object’s owner. `videos` in the analysis project holds the canonical copy the pipeline reads at `{video_id}/{filename}` together with `{video_id}/poster.jpg`, with public read so that a playback URL is permanent and unsigned.

Because the object path begins with the content hash, re-ingesting identical bytes resolves to an object that already exists and the upload is skipped. The service checks for the object before writing rather than relying on overwrite semantics, so a repeated ingest of a large recording costs nothing.

7.4 Analysis Record Store

One JSON document is written per recording, named `{sanitised-filename}_{video_id[:8]}.json`. The filename carries a readable stem so that a directory of records can be inspected directly, and the truncated content hash so that two recordings sharing a name cannot collide. On re-ingest any existing document for the same identifier is removed before the new one is written, so a stale record can never sit alongside a current one under a different name.

The document holds the recording’s identity and media (`video_id`, `video_url`, `storage_path`, `source_url`, `filename`, `poster_url`, `duration`, `size_bytes`), the chunking scheme that produced it (`preset`, `params`, `chunk_config`), the analyzers that ran, the `chunks` array, and the `aggregates` object with the analyzer set they were computed from, so that a stale cache is detectable.

Each entry of `chunks` carries `id`, `start` and `end`, followed by one key per analyzer that produced output for that chunk, whose value is that analyzer’s structured result: `default_video`

holding the scene schema, **people** holding per-person records with their box locations, **ocr** holding the read texts, and so on. An analyzer that was gated out for a chunk contributes no key, so absence is represented naturally rather than as a null. This is the shape a relational schema cannot accommodate without a migration per analyzer.

Two conventions apply throughout. Keys prefixed with an underscore, such as **_frames** and **_detector_labels**, are provenance retained for debugging; they are never embedded, never copied into a vector payload and never returned by the API. And no local filesystem path is ever persisted: the cached file is re-derived from **video_id** on demand and re-downloaded if the cache is cold, so a record remains correct after the data directory moves or the cache is wiped.

7.5 Vector Store

A single Qdrant collection, **chunks**, holds one point per recording, analyzer, chunk span and chunk configuration. Each point carries all five named vectors, **combined** (the whole flattened record, and the default search space), **description**, **people**, **actions** and **objects**, each 384-dimensional and compared by cosine distance. Making the point rather than the vector the unit of storage means a limit of ten returns ten chunks, the payload is stored once, and a filter is evaluated once. A vector space is omitted rather than written empty, so a chunk in which no person was seen is not a candidate in a people-scoped search at all.

The point identifier is a UUID derived deterministically from `sha1(video_id | analyzer_id | start | end | chunk_id)` so that re-ingesting the same chunk upserts in place rather than duplicating. The key is built from the time span rather than a positional index, because chunk 12 of one chunking run is a different moment from chunk 12 of another and a positional identifier would silently rebind a vector to the wrong timeframe. Custom chunking weights are hashed into **chunk.config** for the same reason.

The payload carries everything required to rank, filter and cite a moment, so retrieval never joins back to the record store: the recording identifier and playback URL, the analyzer, chunk configuration and chunk identifier, the time span, the combined text and description, the setting, the people, objects, actions and tags, the speakers and their turns, the texts read from screen, the object detections, the full per-person records, and the person count.

Filter	Payload field	Match	Type
video_ids	video_id	any	<code>list[str]</code>
chunk_ids	chunk_id	any	<code>list[int]</code>
analyzer_ids	extractor_id	any	<code>list[str]</code>
chunk.config	chunk.config	exact	<code>str</code>
objects	objects	any	<code>list[str]</code>
tags	tags	any	<code>list[str]</code>
speakers	speakers	any	<code>list[str]</code>
people	people	any	<code>list[str]</code>
min.people	people_count	$\geq$	<code>int</code>
max.people	people_count	$\leq$	<code>int</code>
after	start	$\geq$	<code>float</code>
before	end	$\leq$	<code>float</code>

The filter set is declared in one place and the API passes a flat dictionary through to it, so a filter added to the declaration is reachable from the API immediately; the previous hand-plumbed parameter list left six store filters silently unreachable. An unrecognised filter name raises an error carrying the nearest valid name, because a filter that is silently ignored returns plausible but wrong results, which is worse than a failure.

The same twelve fields are declared as payload indexes. The person count is indexed specifically because counting is what embeddings cannot do: a query for a mostly empty store retrieved the busiest chunk in the recording, since a description listing five people reads much the same to a vector as one listing two. In embedded mode Qdrant ignores payload indexes and filtering scans instead, which is correct and acceptable at this scale; the declarations are retained so that pointing the same code at a Qdrant server gains real indexes with no change.

Deletion is scoped rather than wholesale. Removing vectors takes an optional chunk configuration and analyzer identifier, because re-running one analyzer must delete only that analyzer's vectors. Deleting by recording alone would destroy every other analyzer's work, so adding one analyzer to an existing recording would silently discard the rest.

7.6 Cross-Store Identity

The content hash is the single join key across all four stores. It is computed while the bytes are being read, which is why the download must complete before the object's own name exists.

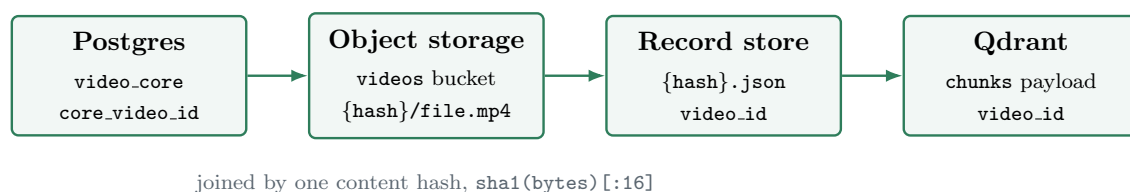


Figure 5: The content hash of the video bytes is the identity shared by every store, which is what makes ingestion idempotent (NFR-10).

Durability differs by store, and the difference is intentional. The record store is authoritative: it holds the analyzer and aggregator output, which is the expensive artifact, and everything else can be rebuilt from it. The vector store is derived, so re-indexing rebuilds it from the records at the cost of local embedding alone, with no API spend and no re-analysis. The media cache is regenerable, so deleting it costs a re-download and never data, which is why no local path is persisted anywhere. Job state is deliberately held in memory and does not survive a restart; the application detects the loss when a poll returns nothing, and either recovers the recording from its persisted record or fails the row explicitly so that re-indexing becomes the route forward.

8 User Interface Design

The interface is organised around a single principle: the operator should never be required to scrub. Every screen terminates in a moment that plays. Figure 6 shows the four principal states of the application: the project workspace, the ingestion dialog, the agent chat with its clip reel and timeline studio, and the per-recording detail view.

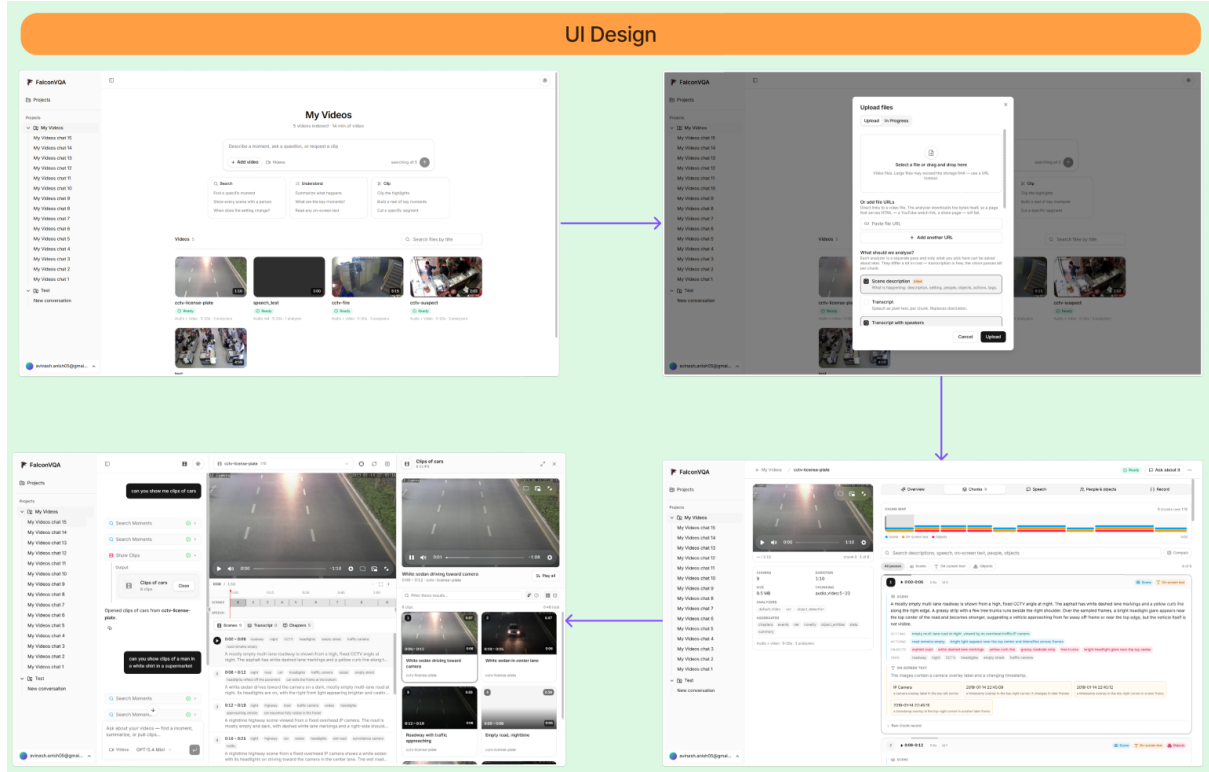


Figure 6: Operator interface. Clockwise from top left: the project workspace and video grid; the ingestion dialog, whose analyzer list is built from the analysis service’s own capability endpoint; the per-recording detail view with its chunk map and tabbed record; and the agent chat with the clip reel artifact panel and the timeline studio.

Three decisions in the interface follow directly from the requirements. Progress is reported per recording rather than as a global spinner, so that a workspace with several recordings being processed at once remains legible (FR-3). A recording that is not yet ready is excluded from search with a stated reason instead of silently returning nothing, because an empty result and an unfinished index are different situations that look identical to the operator. And every result carries its timecode as the primary handle, since the timecode is what turns a search result into footage the operator can watch.

Screen	Operator activity
Landing and documentation	Presents the system and hosts the full documentation: concepts, workspace guides, and the API reference.
Authentication	Registration and sign-in. Every subsequent route is guarded, and every query is restricted to the signed-in account.
Projects	Lists projects, each grouping a set of recordings. Opening one enters its workspace.
Workspace	Adds recordings by file or by link, selects the analyzers and the chunking mode, and observes each recording advance through Uploading, Indexing and Ready. Only ready recordings may be searched, and the interface states this rather than returning an empty result.
Agent chat	Accepts a description or a question, and a selection of recordings to consider. The assistant chooses the retrieval operations, and answers remain concise and cite timecodes. Conversations persist and may be resumed.
Clip reel panel	Opens beside the chat whenever the results are moments. A player is placed above the list of moments; selecting one plays it, and they may be played consecutively as a reel.
Timeline studio	Presents the recording's scenes, transcript and chapters against a timeline that follows the playhead, for reviewing what was indexed in the region being watched.
Recording detail	Five tabs over one recording: Overview for the video-level output, Chunks for the chunk map and per-chunk records, Speech for the transcript, People and objects for linked entities and their timelines, and Record for the raw stored document.

9 Acceptance Test Cases and Test Plan

This section states what has to be true for the system to be accepted, gives the test cases that establish it, and then sets out the plan under which those cases are run: the environment, the test data, the entry and exit criteria, what is automated, and how failures are recorded.

9.1 Acceptance Criteria

Every requirement from Section 3 is restated below as a condition that can be observed to hold or not hold, together with the cases that establish it. A requirement with no case would be a requirement nobody agreed how to check, so the third column is also the traceability matrix.

Req.	Accepted when	Verified by
FR-1	An operator can register, sign in and group recordings into projects, and no account can reach another account's projects, recordings or conversations by any route.	AT-01 to AT-04, AT-06
FR-2	A recording can be added by file or by link, the selected analyzers are the ones that run, and the recording reaches ready .	AT-07, AT-08, AT-09
FR-3	Stage and progress are visible throughout processing, the rest of the workspace stays usable, and a failure explains itself instead of hanging.	AT-11, AT-12, AT-17, AT-18
FR-4	A described event is returned within the leading results, and content that is absent is reported as absent.	AT-19, AT-20, AT-24
FR-5	Restricting by time, by content or by recording removes non-matching results and retains matching ones.	AT-21, AT-22, AT-23
FR-6	An answer is correct against the ground-truth sheet, carries a timecode for every claim, and is refused when the footage does not support it.	AT-27, AT-28, AT-29
FR-7	Selecting a result plays the footage from that moment with no scrubbing, individually or as a reel.	AT-32, AT-33
FR-8	Summary, chapters, events, entities and anomalies are produced for a recording and are legible without watching it.	AT-35, AT-37, AT-38
FR-9	A multi-turn conversation selects the right tools without the recording being named each turn, and survives being reopened.	AT-30, AT-31
FR-10	The stored record for any chunk is retrievable, and the timeline follows the playhead.	AT-34, AT-36
FR-11	A recording can be re-examined at another depth without being re-added, and earlier results survive.	AT-14, AT-15
FR-12	Every retrieval capability is reachable over the documented API, and a client can be built from the capability endpoint alone.	AT-26, AT-39, AT-41
NFR-1	Search returns within 2s and an answer within 15s on a warm instance.	PT-1, PT-2
NFR-2	A recording becomes searchable within roughly three times its own duration.	PT-3
NFR-3	Stage and progress refresh at least every 5s during ingestion.	AT-11, PT-4
NFR-4	Playback of a selected moment begins within 2s, including the second and later moments of a reel.	AT-32, AT-33, PT-5
NFR-5	No answer is produced without citations, and an unsupported question is declined.	AT-27, AT-29
NFR-6	Silent, cut-free, fixed-camera footage yields multiple usable chunks rather than one span.	AT-16
NFR-7	Deselecting an analyzer avoids its cost, a gated frame incurs no model call, and reviewing existing results incurs none.	AT-09, AT-26, AT-38, PT-3, PT-6
NFR-8	A new analyzer appears in the API and the ingest dialog after one module and one registry line, with no other edit.	AT-10, AT-40
NFR-9	Isolation holds at the database and at the agent boundary, including against a fabricated identifier.	AT-03 to AT-06
NFR-10	The same file submitted twice is recognised as one recording and is neither stored nor processed twice.	AT-13

9.2 Case Format and Severity

Each case below gives its precondition, the procedure to follow, and the result that constitutes a pass. Cases are grouped by the area they exercise and are intended to be run in order within a group, since later cases in a group often depend on state the earlier ones create. The fixtures named in the procedures are specified in Section 9.11, and the pass conditions for search and question cases are scored against the ground-truth sheets described there rather than against a tester's impression at the time.

Severity is assigned to a failure as follows. **Critical** means the system cannot be used for its purpose, or a tenant can reach another tenant's data. **Major** means a stated requirement is not met but a workaround exists. **Minor** means the result is correct but the presentation or the timing is not what was intended.

9.3 Test Cases: Accounts and Tenancy

ID	Case and precondition	Procedure	Expected result
AT-01	Registration and sign-in. No account exists for the address used.	1) Register with an address and password. 2) Sign out. 3) Sign in again. 4) Request a guarded route while signed out.	The account is created and the session is established. Step 4 redirects to sign-in rather than rendering the route or returning data.
AT-02	Project creation and listing. Account A signed in.	1) Create two projects. 2) Return to the project list. 3) Open one. 4) Rename it.	Both projects appear, newest first. Opening one enters its workspace. The rename is reflected in the list without a reload.
AT-03	Cross-account isolation. Account A owns a project holding a ready recording and a saved conversation.	1) Sign in as B. 2) Inspect the project list, the workspace and the conversation list.	B sees none of A's projects, recordings, conversations or messages. B's own lists are empty rather than populated with A's rows.
AT-04	Access by identifier. A's project, recording and conversation identifiers are known. Signed in as B.	1) Request each of A's identifiers directly by URL. 2) Repeat against the client's own API routes.	Every request is refused or returns empty. No row belonging to A is returned by any route, which is row-level security rather than interface filtering.
AT-05	Agent scoping against a fabricated identifier. B signed in, conversation attached to B's project.	1) Ask the agent a question naming a recording identifier that belongs to A.	The identifier is filtered out by the project scoping function. The agent answers from B's own recordings or states that the recording is not in this project. It never returns content from A's footage.
AT-06	Cascading delete. A owns a project with recordings, a conversation and messages.	1) Delete the project. 2) Query for the child rows directly.	Recordings, conversations and messages belonging to the project are gone. No orphaned row remains.

9.4 Test Cases: Ingestion and Processing

ID	Case and precondition	Procedure	Expected result
AT-07	Add a recording by upload. Empty project.	1) Open the ingest dialog. 2) Select cctv-shopping . 3) Accept the default analyzers and chunking mode. 4) Submit and wait.	The recording appears immediately with a poster frame, advances through its stages, and reaches ready . Chunk count and duration are populated.
AT-08	Add a recording by link. A reachable direct video URL.	1) Choose the link tab. 2) Paste the URL. 3) Submit and wait.	The service fetches the file, hashes it, and the recording reaches ready with the same behaviour as an upload. source.url and playback.url both resolve.
AT-09	Analyzer selection is honoured. Ingest dialog open.	1) Deselect people and text recognition, leaving scene and transcription. 2) Ingest. 3) Inspect the stored record and the recording's analyzer list once ready.	Only the selected analyzers appear in the produced list. No chunk carries a key for a deselected analyzer, and no cost was incurred for one.
AT-10	Analyzer list is discovered, not hard-coded. Analysis service running.	1) Open the ingest dialog. 2) Compare its analyzer list against the capability endpoint's response.	The two agree exactly. The dialog builds its list from the endpoint rather than from a list compiled into the client.
AT-11	Progress is visible and the interface stays usable. Ingestion of a 5-minute recording in progress.	1) Observe the recording card throughout. 2) While it processes, open another recording and run a search.	Stage and progress update at least every 5 seconds and move monotonically through fetching, chunking, analyzing, indexing and aggregating. The rest of the workspace remains responsive.
AT-12	Only ready recordings are searchable. A recording mid-ingest.	1) Attempt to include it in a search or a question.	The interface states that the recording is still being processed. It does not return an empty result set that would read as nothing found.
AT-13	Idempotent ingestion. cctv-shopping already ingested into project P.	1) Add the identical file again to project P. 2) Add it to a second project Q. 3) Inspect the object storage path and the record store.	In P the file is recognised as the same recording by its content hash: it is not uploaded, re-analysed or duplicated. In Q a separate row exists, but both rows share one core.video.id , one stored object and one record document.
AT-14	Re-examine at a different depth. A recording indexed at audio-video:5-20 .	1) Re-index with a different chunking configuration. 2) Search against each configuration in turn.	Both chunkings coexist. Vectors from the first are not overwritten, and a search scoped to one configuration returns only that configuration's chunks.
AT-15	Adding one analyzer to an existing recording. A recording indexed without text recognition.	1) Re-run with text recognition added. 2) Inspect the other analyzers' chunk records and vectors.	Text recognition output appears. Every previously produced analyzer's records and vectors are intact, because deletion is scoped to the analyzer being re-run.
AT-16	Silent, cut-free footage. cctv-shopping , which has neither speech nor hard cuts.	1) Ingest with the default preset. 2) Inspect the resulting chunk boundaries and descriptions.	Multiple chunks are produced, not one span covering the whole recording. Weight redistribution puts the load on the signals that fired. Descriptions differ between chunks.
AT-17	Failed ingestion explains itself. A URL that does not resolve to a video.	1) Submit it. 2) Observe the recording card.	The recording reaches failed , the reason is retained on the row and shown, and the failure does not leave the workspace in a loading state.
AT-18	Lost job recovery. Ingestion in progress.	1) Restart the analysis service mid-ingest. 2) Let the client poll.	The client detects that the job no longer exists and either recovers the recording from its persisted record or fails the row explicitly, so that re-indexing is available. It does not poll indefinitely.

9.5 Test Cases: Search and Filtering

ID	Case and precondition	Procedure	Expected result
AT-19	Search by description. <code>cctv-shopping</code> ready, ground-truth sheet written.	1) Take five events from the sheet. 2) Search for each in natural language, using wording that does not repeat the stored description. 3) Record the rank of the correct moment.	For each query the correct moment is within the top three results, and its timecode is within one chunk of the ground-truth timecode.
AT-20	Vector space scoping. Same fixture.	1) Search for a person by clothing, scoped to the people vector space. 2) Repeat against the combined space.	The people-scoped search ranks the correct person higher, because the query is matched against a short person description rather than the whole chunk record.
AT-21	Time filter. Same fixture.	1) Search for an event known to occur late in the recording. 2) Repeat restricted to the first 60 seconds.	The unrestricted search returns the moment. The restricted search excludes it, and every returned span lies inside the requested window.
AT-22	Content filters. Same fixture.	1) Filter by an object known to appear. 2) Filter by a minimum person count of three. 3) Filter by a speaker on the multi-speaker fixture.	Each filter removes non-matching chunks and retains matching ones. The person count filter answers a counting query that description similarity alone gets wrong.
AT-23	Recording scope. Two ready recordings in one project.	1) Search with both selected. 2) Search with one selected.	The scoped search returns results only from the selected recording.
AT-24	Absent content is reported as absent. Any ready recording.	1) Search for something the ground-truth sheet confirms is not present.	The score threshold suppresses the nearest neighbours and the result is empty, rather than the top-ranked unrelated chunk being presented as a match.
AT-25	Unknown filter is rejected. Direct API access.	1) Issue a search carrying a misspelled filter name.	The request fails with an error naming the nearest valid filter. The filter is not silently ignored, which would return plausible but wrong results.
AT-26	Response detail levels. Direct API access.	1) Issue the same search at <code>minimal</code> , <code>standard</code> and <code>full</code> . 2) Compare the payloads.	All three return the same ranked moments. The payload grows with the level, and <code>minimal</code> is small enough to be economical for an agent.

9.6 Test Cases: Question Answering and Conversation

ID	Case and precondition	Procedure	Expected result
AT-27	Answer with citations. cctv-shopping ready.	1) Ask five questions drawn from the ground-truth sheet. 2) Check each claim against the sheet. 3) Follow each citation.	Every answer is factually correct against the sheet. Every claim carries a timecode reference, and following a reference plays footage that supports the claim.
AT-28	Routing to the right aggregate. Same fixture, aggregates computed.	1) Ask about a specific person. 2) Ask what was unusual. 3) Ask how busy the store was. 4) Inspect the routing recorded on each response.	The person question routes to entities, the unusual question to novelty, the busyness question to statistics. Routing is reported so that the answer can be traced.
AT-29	Unanswerable question. Any ready recording.	1) Ask about something the footage does not show, phrased as though it does.	The system states that the footage does not support an answer. It does not synthesise a plausible answer, and it does not cite an unrelated moment as evidence.
AT-30	Multi-turn conversation. Project with two ready recordings.	1) Ask an opening question. 2) Ask three follow-ups that refer back by pronoun without naming the recording. 3) Observe the tools the agent selects.	Context carries across turns. The agent chooses search, question or inspection operations appropriately without the recording being named again.
AT-31	Conversation persistence. A conversation with several turns.	1) Leave the page. 2) Sign out and back in. 3) Reopen the conversation.	Every turn is restored in order, including tool calls, results and artifacts. Continuing the conversation retains the earlier context.

9.7 Test Cases: Review, Playback and Inspection

ID	Case and precondition	Procedure	Expected result
AT-32	Play a result at its moment. A search returning several moments.	1) Select a result. 2) Observe where playback begins.	Playback starts at the result's start timecode with no manual scrubbing, and stops at its end.
AT-33	Clip reel. Several results drawn from one recording.	1) Open the clip reel panel. 2) Play the moments consecutively.	The moments play one after another. The MP4 is loaded once and seeked, so the second and later moments start without a fresh load.
AT-34	Timeline studio. A ready recording open.	1) Play the recording. 2) Watch the scene, transcript and chapter tracks. 3) Scrub to a new position.	The timeline follows the playhead, and the region shown corresponds to what was indexed at that position. Scrubbing moves the tracks with it.
AT-35	Video-level review. cctv-shopping ready with aggregates computed.	1) Open the recording detail view. 2) Read the summary, chapters, events, entities and anomalies without watching the footage. 3) Compare against the ground-truth sheet.	Each aggregate is produced and legible. The summary and chapters describe what actually occurs. Anomalies flag moments the sheet marks as unusual.
AT-36	Index inspection. Same fixture.	1) Open the chunk map. 2) Select a chunk. 3) Read its stored record across the tabs.	The description, on-screen text, people, objects and speech recorded for that chunk are all retrievable, including the raw stored document.
AT-37	Entity linking within a recording. A fixture in which one person appears several times.	1) Open the people and objects tab. 2) Follow one person's timeline.	Separate sightings of the same person are linked into one individual with an ordered account of what they did. Two people visible in the same chunk are never merged.
AT-38	Aggregate caching. Aggregates already computed.	1) Reopen the detail view. 2) Re-request the aggregates.	Cached results are returned with no re-analysis and no model spend. Changing the analyzer set marks the cache stale and triggers recomputation.

9.8 Test Cases: Programmatic Access

ID	Case and precondition	Procedure	Expected result
AT-39	Client built from the capability endpoint alone. A ready recording; no access to the source.	1) Request the capability endpoint. 2) Using only its response, construct and issue a valid search with a filter.	The endpoint describes every analyzer, vector field, filter and aggregator needed. The constructed request succeeds.
AT-40	A new analyzer requires no client change. Analysis service source available.	1) Register a trivial analyzer as one module and one registry line. 2) Restart the service. 3) Reload the ingest dialog and the capability endpoint.	The analyzer appears in both without any edit to ingestion, storage, the API or the client.
AT-41	Documented API matches behaviour. Documentation site deployed.	1) Take each endpoint from the API reference. 2) Issue it against the running service.	Every documented endpoint exists, accepts the documented parameters and returns the documented shape.

9.9 Non-Functional Verification

The requirements that state a figure are verified by measurement rather than by observation. Each is measured on a warm instance, repeated ten times, and reported as the median together with the slowest of the ten, because a target met on average but missed regularly is not met.

ID	Measurement	Method	Target
PT-1	Search latency	Time from request to ranked moments at the API, and to rendered results in the browser, over ten distinct queries against a 5-minute recording.	$\leq 2\text{ s}$ (NFR-1)
PT-2	Answer latency	Same method, over ten questions spanning the routed aggregates.	$\leq 15\text{ s}$ (NFR-1)
PT-3	Processing time	Wall-clock from submission to ready , divided by the recording's own duration, with the default analyzer set.	$\leq 3\times$ duration (NFR-2)
PT-4	Progress refresh	Interval between successive stage or progress updates observed on the recording card during a full ingest.	$\leq 5\text{ s}$ (NFR-3)
PT-5	Playback latency	Time from selecting a result to the first frame at that time-code, for a first moment and for a subsequent moment in the same reel.	$\leq 2\text{ s}$ (NFR-4)
PT-6	Response size	Token count of an identical five-hit search at each detail level, so that the cost of the agent path is known rather than assumed.	Recorded

Cost control (NFR-7) is verified alongside PT-3 by counting model calls: a re-run of an aggregator over stored records must issue no analyzer calls, and a gated frame must issue none at all. Robustness (NFR-6) is not a figure and is covered by AT-16.

9.10 Test Environment

The cases above are run against a deployed instance over HTTP and through the browser, not against modules in isolation, because several of the defects found during development (stale background jobs, broken answer synthesis, media paths that resolved locally but not remotely)

were only visible once the whole system was running. Unit-level behaviour of individual analyzers and detectors is out of scope here.

Element	Configuration under test
Analysis service	Python 3.13 and FastAPI on a CUDA-capable host, RTX 4060 with 8 GB VRAM, 16 GB system RAM. Served over HTTP with Uvicorn, not with the reload server.
Client	Next.js production build, deployed on Vercel, pointed at the analysis service by environment variable.
Application data	A dedicated Supabase project with row-level security enabled and the schema migrations applied from a clean state.
Vector store	Qdrant in embedded mode, backed by a data directory that is emptied before a full test pass.
Browsers	Chrome and Firefox, current stable releases, on a desktop viewport.
Accounts	Two registered accounts, A and B, used for every isolation test. Neither is an administrator.
Model providers	Live API keys for the vision and language models, and a Hugging Face token for the gated diarization model. Tests are not run against mocked providers, because provider behaviour is part of what is being accepted.

Latency cases are run warm, meaning model weights are already resident and the media cache holds the recording. Cold-start timings are recorded separately and reported as such rather than being counted against the response targets, since a first request pays for model loading that no subsequent request repeats.

9.11 Test Data

The same six recordings are used throughout, so that a result observed in one case can be compared against the same footage in another. Each fixture was chosen because it isolates a property the others do not exercise.

Fixture	Content	Property it isolates
cctv-shopping	5 minutes, 1280×720, fixed camera, no speech	The principal fixture. Neither cuts nor audio, which is the case general-purpose tools fail on. Used for chunking, scene, people, object and text cases.
chernobyl-audio-video	60 seconds, several speakers	The only fixture in which speaker change, silence and speaker-attributed transcription all fire. Used for every audio path.
cctv-license-plate	Short fixed-camera clip, vehicle plates	On-screen text recognition against low-contrast, small-format text.
cctv-fire, cctv-car-fire	Short fixed-camera clips	Novelty detection and event extraction, where the anomalous moment is known in advance.
cctv-suspect	Short fixed-camera clip, suspected theft	Person description and the moments used in clip reel testing.
Extended set (planned)	Longer footage, multiple cameras, one shared subject	Required for cross-recording person linking, which cannot be tested on footage that shares no people. Recorded as a known gap in Section 6.

For each fixture, a ground-truth sheet is written before testing begins: a list of what occurs in the recording and the timecode at which it occurs. Search and question cases are scored against that sheet, so that a re-run of the same case on a later build is comparable.

9.12 Entry, Exit and Suspension

Criterion	Condition
Entry	The full pipeline runs end to end on at least one fixture; the client is deployed and reaches the analysis service; migrations are applied; both test accounts exist; ground-truth sheets are written.
Exit	Every Critical and Major case passes. No Critical or Major defect is open. Any open Minor defect is recorded with the reason it is accepted. Every requirement in Section 9.1 traces to at least one passing case.
Suspension	Testing stops if the analysis service is unreachable, a model provider is returning errors, or a defect prevents ingestion from completing, since almost every later case depends on an indexed recording.
Resumption	Testing resumes from the start of the group in which it was suspended, not from the individual case, because a suspended run may have left partial state.

9.13 Test Automation

Not every case is worth automating. The ones that are worth automating are those run repeatedly whose pass condition a machine can check: an HTTP status, a rank, a count, a returned identifier. The ones that are not are those whose pass condition is a judgement about footage, and those stay manual. The split is set out below.

Layer	Tooling	What it covers
API regression	Postman collection with an environment file, run headless by Newman	Every endpoint in the API reference, with assertions on status, response shape and the capability endpoint's contents. Covers AT-25, AT-26, AT-39 and AT-41 directly, and builds on the collection already maintained alongside the service documentation.
Retrieval scoring	A Python harness over the ground-truth sheets	Issues the recorded queries against a live server and asserts the rank of the expected moment and the timecode it resolves to. This is what turns AT-19 to AT-24 from a subjective judgement into a number comparable across builds.
Pipeline benchmarks	The existing <code>scripts/bench.py</code>	Per-stage timings for detectors, fusion, chunking and export on a fixture, which is the instrument PT-3 reads. Extended to record search and answer latency for PT-1 and PT-2.
Tenancy checks	SQL assertions run as each test account	Attempts a read of the other account's rows in every table and asserts that nothing is returned. Covers AT-03, AT-04 and AT-06, which are the cases most costly to get wrong and least interesting to repeat by hand.
Browser flows	Playwright	Sign-in, project creation, ingest to ready , search, and playback starting at the expected timecode. Covers the happy path of AT-01, AT-02, AT-07, AT-11 and AT-32.
Manual only	Structured checklist against the ground-truth sheets	Whether a description is a fair account of the footage, whether a summary reads sensibly, whether linked people are the same person, and whether an answer is genuinely supported by the moment it cites. These are AT-27, AT-28, AT-35 and AT-37, and a machine scoring them would only be checking one model against another.

Two constraints shape how far automation is taken. The pipeline needs a GPU and live model providers, so the suite cannot run on a hosted runner without one, and a full ingest of the

principal fixture costs real money on every run. The suite is therefore split: the API regression, tenancy checks and browser flows run against a pre-indexed fixture and are cheap enough to run on every change, while the retrieval scoring and benchmark runs, which require a fresh ingest, are run before a release rather than on every commit.

9.14 Defect Recording and Reporting

Each executed case is recorded with its identifier, the build and fixture it ran against, the observed result, and a pass or fail verdict. A failure is logged with its severity, the request or interaction that produced it, and whatever the system reported at the time. Because ingestion runs as a background job, a failure during processing is recorded together with the stage the job had reached and the reason retained on the recording row, since that is usually the only account of what went wrong.

A defect is closed when the case that found it passes on a later build and the cases around it in the same group still pass. A fix that repairs one case and breaks its neighbour is not a fix, which is why the resumption rule above restarts a group rather than a single case.

10 Execution Plan

10.1 Schedule

Development proceeds in eight phases. Each phase is validated end to end before the next is begun, so that a defect in an earlier stage is not discovered through the behaviour of a later one.

Phase	Stage	Work
Phase 1	Chunking	Boundary detectors, weighted fusion, the three chunking modes and the duration bounds. Validated on cut-free footage before anything was built upon it.
Phase 2	Analysis	Scene, people, object, text and speech analyzers; frame sampling; detector gating; and the registry that renders them interchangeable.
Phase 3	Indexing and retrieval	Local embedding, the five named vectors, the typed filter set, response detail levels and the search endpoint.
Phase 4	Aggregation	Video-level output, dependency ordering, caching, and person and object entity linking.
Phase 5	Question answering	Routing of questions to the aggregates capable of answering them, and answer synthesis with mandatory citations.
Phase 6	Application tier	Authentication and row-level security, projects and workspace, upload and status reconciliation, the agent tool layer, clip reel and timeline studio, and the per-recording detail view.
Phase 7	Documentation	The MDX documentation site, covering concepts, workspace guides and the complete API reference, together with the public landing pages.
Phase 8	Hardening and reporting	Execution of the acceptance test cases in Section 9 against a live server, the automated suites that support them, error handling, deployment, and this document.

10.2 Distribution of Work

The project is undertaken by two members. The division follows the pipeline: everything from a recording arriving to its being chunked and stored on one side, and everything from a chunk being analysed to a question being answered on the other.

Member	Responsibility
Avinash Anish	Application tier and ingestion. The entire interface, covering authentication, projects and workspace, upload and status, agent chat, clip reel panel, timeline studio and the recording detail view, together with the agent tool layer and its project scoping. On the pipeline side, ingestion: source handling, object storage and caching, content-addressed identity, the four boundary detectors, weighted fusion and the three chunking modes, and the background job and progress reporting behind them.
Vaivaswat Verma	Analysis service and retrieval. Everything downstream of chunking: the analyzers with their frame sampling and detector gating, embedding and the vector store, filters and detail levels, the video-level aggregators including person and object entity linking, and question routing and answer synthesis. Owns the HTTP API through which these are exposed.
Shared	Architecture and design decisions, acceptance testing against a live server, deployment, and documentation.